In [1]:
```
# Behzad Ghabaei
# CS 82A - Data Science
# module5_lab3.ipynb
# custdata.tsv
# Text Book Problem 5.5
# Age, income, vehicles: what correlates?
# Instructor Seno
# 10/1/2026
```

In [2]:
```
import pandas as pd
df = pd.read_csv("custdata.tsv", sep="\t")
```

Lets import the dataset and run four commands first.

In [3]:
```
df.describe()
```

Out[3]:

	custid	income	num.vehicles	age
count	1.000000e+03	1000.000000	944.000000	1000.000000
mean	6.984997e+05	53504.771000	1.916314	51.699815
std	4.135083e+05	65478.065729	1.101618	18.863433
min	2.068000e+03	-8700.000000	0.000000	0.000000
25%	3.456668e+05	14600.000000	1.000000	38.000000
50%	6.934030e+05	35000.000000	2.000000	50.000000
75%	1.044606e+06	67000.000000	2.000000	64.000000
max	1.414286e+06	615000.000000	6.000000	146.680197

Age min is at 0 which doesn't exist, age max at 146, which is hard to believe.

Num.vehicles min and max look true.

Income max is ok, but min is a negative number, looking like an impossible number.

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   custid        1000 non-null   int64
 1   sex           1000 non-null   str
 2   is.employed   672 non-null    object
 3   income        1000 non-null   int64
 4   marital.stat  1000 non-null   str
 5   health.ins    1000 non-null   bool
 6   housing.type  944 non-null    str
 7   recent.move   944 non-null    object
 8   num.vehicles  944 non-null    float64
 9   age           1000 non-null   float64
 10  state.of.res  1000 non-null   str
dtypes: bool(1), float64(2), int64(2), object(2), str(4)
memory usage: 79.2+ KB
```

NaN empty data points listed in num.vehicles, 1000 - 944 = 56

In [5]:
```
df.head()
```

Out[5]:

	custid	sex	is.employed	income	marital.stat	health.ins	housing.type	recent.move	num.vehicles	age
0	2068	F	NaN	11300	Married	True	Homeowner free and clear	False	2.0	49.0
1	2073	F	NaN	0	Married	True	Rented	True	3.0	40.0
2	2848	M	True	4500	Never Married	False	Rented	True	3.0	22.0
3	5641	M	True	20000	Never Married	False	Occupied with no rent	False	0.0	22.0
4	6369	F	True	12000	Never Married	True	Rented	True	1.0	31.0

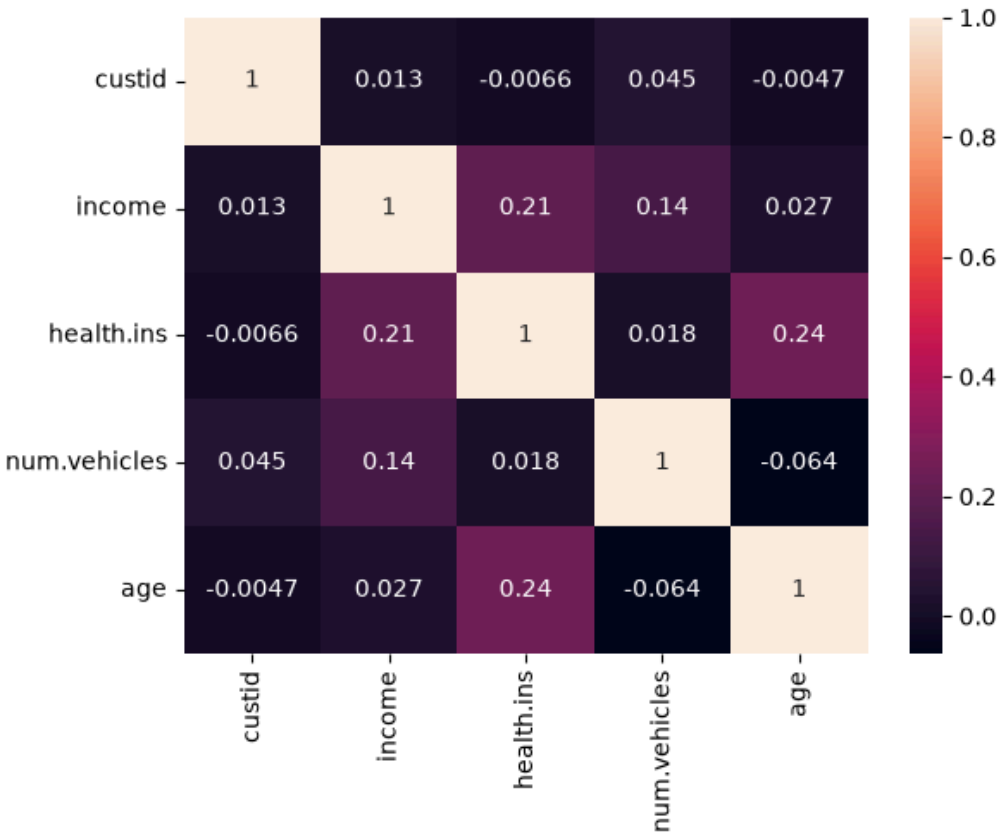
In [6]:
```
df.shape
```

Out[6]:
```
(1000, 11)
```

Lets also take a look at the heapmap for correlations before cleaning the data.

In [7]:
```
import seaborn as sns
sns.heatmap(df.corr(numeric_only=True), annot=True)
```

Out[7]:
```
<Axes: >
```



No powerful linear relationships or correlations are present.

Drop only invalid ages and income, and compare before and after

In [8]:
```
# drop row missing data
#df = df.dropna(subset=["num.vehicles"])
#df = df.dropna(subset=["age"])
#df = df.dropna(subset=["income"])

target_cols = ["age", "income", "num.vehicles"]
df_baseline = df.dropna(subset=target_cols)
```

Correlation (Before Cleaning)

In [9]:
```
print(df_baseline[target_cols].corr(method="pearson"))
```

```
                   age    income  num.vehicles
age           1.000000  0.017142     -0.063666
income        0.017142  1.000000      0.139198
num.vehicles -0.063666  0.139198      1.000000
```

DROP STRANGE AGES & NEGATIVE INCOME

In [10]:
```
valid_age = (df_baseline["age"] > 0) & (df_baseline["age"] < 100)
```

impossible negative incomes

In [11]:
```
valid_income = df_baseline["income"] >= 0
```

a clean dataset

In [12]:
```
df_cleaned = df_baseline[valid_age & valid_income]
```

Cleaned Correlation Matrix

In [13]:
```
print(df_cleaned[target_cols].corr(method="pearson").round(3))
```

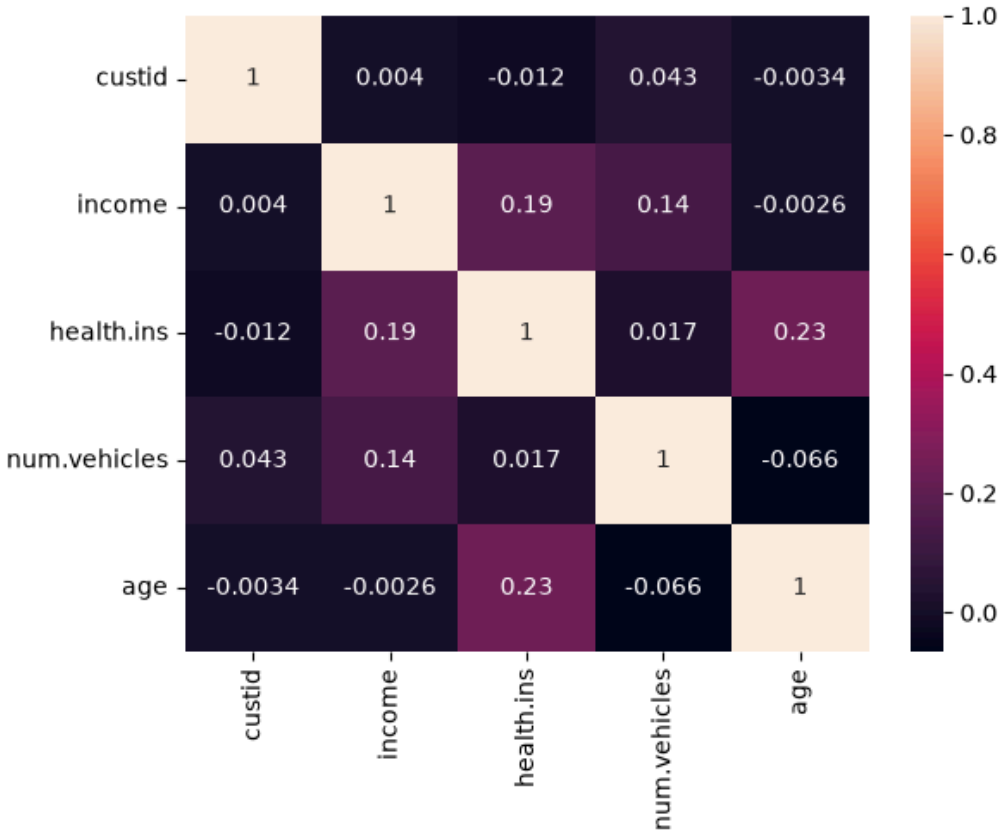
```
                age  income  num.vehicles
age           1.000  -0.003        -0.066
income       -0.003   1.000         0.138
num.vehicles -0.066   0.138         1.000
```

In [14]:
```
print(f"\nFinal Cleaned Dataset Row Count: {len(df_cleaned)}")
```

```
Final Cleaned Dataset Row Count: 932
```

In [15]:
```
import seaborn as sns
sns.heatmap(df_cleaned.corr(numeric_only=True), annot=True)
```

Out[15]: <Axes: >



In [16]:
```
df_cleaned.info()
```

```
<class 'pandas.DataFrame'>
Index: 932 entries, 0 to 999
Data columns (total 11 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   custid         932 non-null    int64
 1   sex            932 non-null    str
 2   is.employed    655 non-null    object
 3   income         932 non-null    int64
 4   marital.stat   932 non-null    str
 5   health.ins     932 non-null    bool
 6   housing.type   932 non-null    str
 7   recent.move    932 non-null    object
 8   num.vehicles   932 non-null    float64
 9   age            932 non-null    float64
 10  state.of.res   932 non-null    str
dtypes: bool(1), float64(2), int64(2), object(2), str(4)
memory usage: 81.0+ KB
```

Below, ages that are 0, should be none.

In [17]:
```
df_cleaned[df_cleaned["age"] <= 0 ]
```

Out[17]:

	custid	sex	is.employed	income	marital.stat	health.ins	housing.type	recent.move	num.vehicles	age	s

Below, no list of ages that are more than 125 years old.

In [18]:
```
df_cleaned[df_cleaned["age"] > 125 ]
```

Out[18]:

custid	sex	is.employed	income	marital.stat	health.ins	housing.type	recent.move	num.vehicles	age	s

Below we have no negative income figure.

In [19]:
```
df_cleaned[df_cleaned["income"] < 0 ]
```

Out[19]:

custid	sex	is.employed	income	marital.stat	health.ins	housing.type	recent.move	num.vehicles	age	s

There are no negative vehicles.

In [20]:
```
df_cleaned.describe()
```

Out[20]:

	custid	income	num.vehicles	age
count	9.320000e+02	932.000000	932.000000	932.000000
mean	6.959797e+05	55949.303648	1.914163	51.422747
std	4.138150e+05	66040.083781	1.103229	16.803452
min	2.068000e+03	0.000000	0.000000	19.000000
25%	3.407890e+05	17000.000000	1.000000	39.000000
50%	7.009235e+05	37000.000000	2.000000	50.000000
75%	1.042425e+06	70000.000000	2.000000	63.000000
max	1.414286e+06	615000.000000	6.000000	93.000000

In [21]:
```
df_cleaned.info()
```

```
<class 'pandas.DataFrame'>
Index: 932 entries, 0 to 999
Data columns (total 11 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   custid        932 non-null    int64
 1   sex           932 non-null    str
 2   is.employed   655 non-null    object
 3   income        932 non-null    int64
 4   marital.stat  932 non-null    str
 5   health.ins    932 non-null    bool
 6   housing.type  932 non-null    str
 7   recent.move   932 non-null    object
 8   num.vehicles  932 non-null    float64
 9   age           932 non-null    float64
 10  state.of.res  932 non-null    str
dtypes: bool(1), float64(2), int64(2), object(2), str(4)
memory usage: 81.0+ KB
```

The final count sum of data points, in this cleaned dataset.

In [22]:
```
df_cleaned.shape
```

Out[22]:
```
(932, 11)
```

In [23]:
```
df_cleaned.head()
```

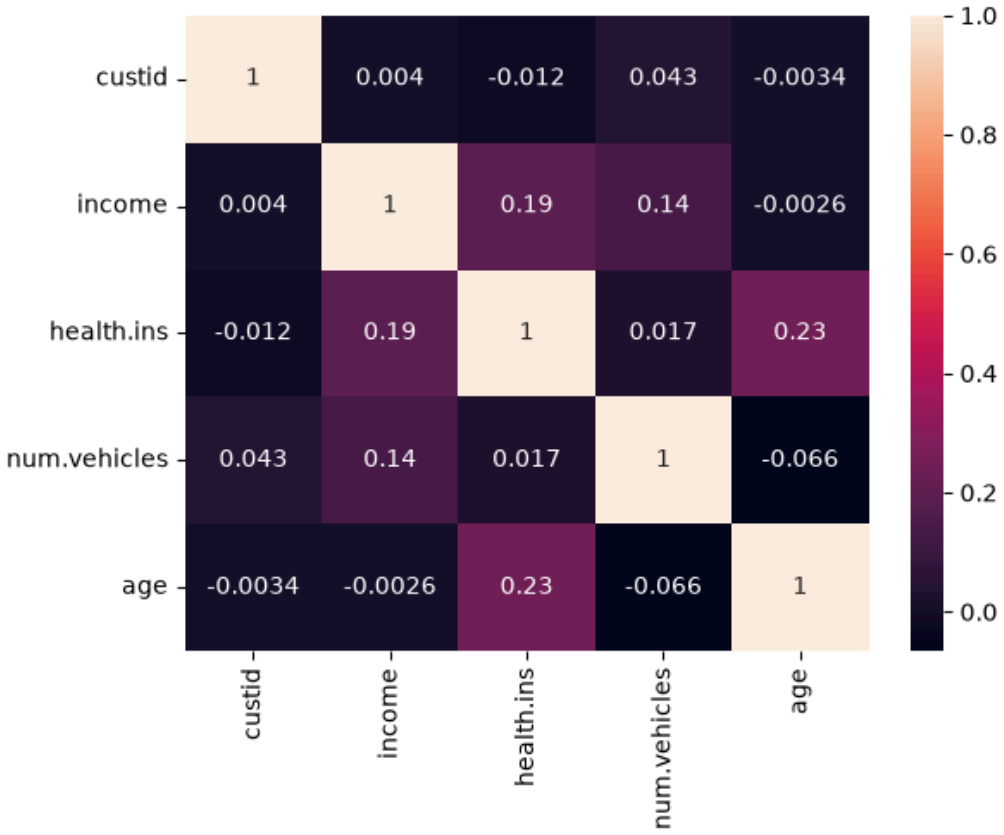
Out[23]:

	custid	sex	is.employed	income	marital.stat	health.ins	housing.type	recent.move	num.vehicles	age
0	2068	F	NaN	11300	Married	True	Homeowner free and clear	False	2.0	49.0
1	2073	F	NaN	0	Married	True	Rented	True	3.0	40.0
2	2848	M	True	4500	Never Married	False	Rented	True	3.0	22.0
3	5641	M	True	20000	Never Married	False	Occupied with no rent	False	0.0	22.0
4	6369	F	True	12000	Never Married	True	Rented	True	1.0	31.0

Seaborn heatmap will display any correlations that may exist. We want a number preferable from +.5 to +1 or -.5 to -1.

In [24]:
```
import seaborn as sns
sns.heatmap(df_cleaned.corr(numeric_only=True), annot=True)
```

Out[24]: <Axes: >



In [25]:
```
print(df_cleaned["age"].corr(df_cleaned["income"]))
print(df_cleaned["age"].corr(df_cleaned["num.vehicles"]))
print(df_cleaned["income"].corr(df_cleaned["num.vehicles"]))
```

```
-0.002644320057175042
-0.06629498011172023
0.13785873187757497
```

After looking at the cleaned data heatmap of Pearson's r correlations, there are no relationships found to be strong enough. The correlation between age, income, and

number of vehicles, is not present.

"Age" vs. "Income" is a weak correlation, filtering drops it closer to zero (-0.003) because A customer's age has no linear correlation with their annual income.

"Age" vs. "Number Vehicles" is another weak relationship. (-0.066). Age does not play a significant linear role in predicting a customer's vehicle count.

"Income" vs. "Number Vehicles" is a weak relationship, weak positive relationship (0.138). There is a minor positive association indicating that higher-income households have a slight tendency to own more vehicles.

Although the datset was enormous and the variables used often could return strong economic relationships, the dataset does not provide a strong enough correlation between these three variables. The ages did not show younger participants as being any wealthier or poorer that older, when looking at age vs. income.

Cleaning Log

Omitted 56 rows with missing (NaN) vehicle counts

Filter Conditions: Applied constraints dropping 9 rows with impossible tracking ages (≤ 0 or ≥ 100) and 3 records with negative s income.

Final Dataset Size: dataset of 932 clean rows.

Behzad Ghabaei - End Of Lab

In [ ]: